Lecture 17: Pretraining

Instructor: Stephanie Schoch

CSCI 680: Natural Language Processing
Department of Computer Science, College of William & Mary
October 19, 2026

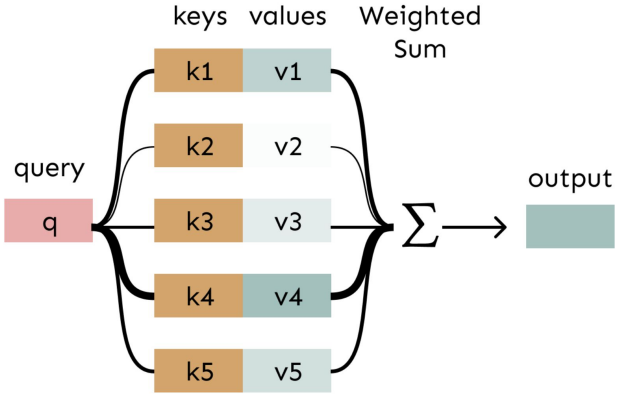
WILLIAM & MARY

Many slides are adapted from Jurafsky & Martin (3rd ed.) & Chris Manning. & John Hewitt

Lecture Outline

- Recap
- Transformers: Putting it All Together
- Pretraining: Data, Architectures, & Objectives

Self-Attention



Let $\mathbf{w}_{1:N}$ be a sequence of words in vocabulary V

For each w_i, let $\mathbf{x}_i = \mathbf{E}_{w_i,:} \in \mathbb{R}^{1\times d}$, where

$\mathbf{E} \in \mathbb{R}^{|\mathcal{V}|\times d}$ is an embedding matrix.

1. Transform each word embedding with weight matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$

$$\mathbf{W}^Q \in \mathbb{R}^{d\times d_k} \qquad \mathbf{W}^K \in \mathbb{R}^{d\times d_k} \qquad \mathbf{W}^V \in \mathbb{R}^{d\times d_v}$$

$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W}^Q \in \mathbb{R}^{1\times d_k} \qquad \mathbf{k}_i = \mathbf{x}_i\mathbf{W}^K \in \mathbb{R}^{1\times d_k} \qquad \mathbf{v}_i = \mathbf{x}_i\mathbf{W}^V \in \mathbb{R}^{1\times d_v}$$

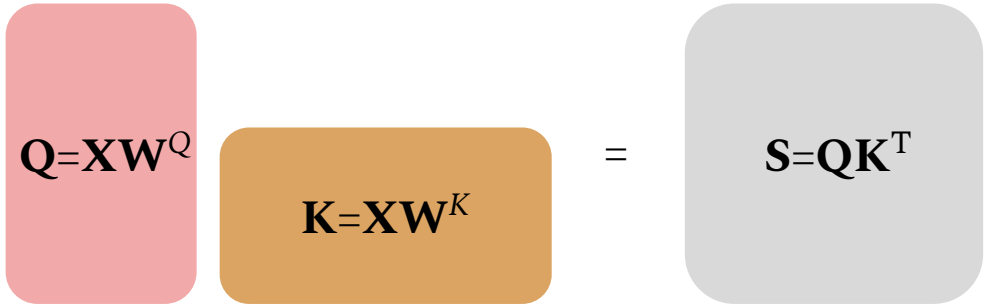
2. Compute similarities between keys and queries; normalize with softmax

$$e_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \qquad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'=1}^{N}\exp(e_{ij'})}$$

3. Compute output for each word as weighted sum of values: $\mathbf{o}_i = \sum_{j=1}^{N}\alpha_{ij}\mathbf{v}_j \in \mathbb{R}^{1\times d_v}$

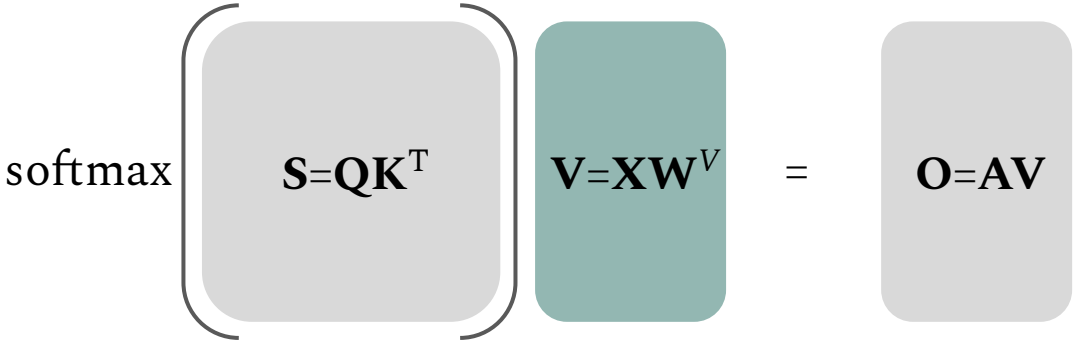
Self-Attention: Matrix Multiplications

First, take the query-key dot products in one matrix multiplication



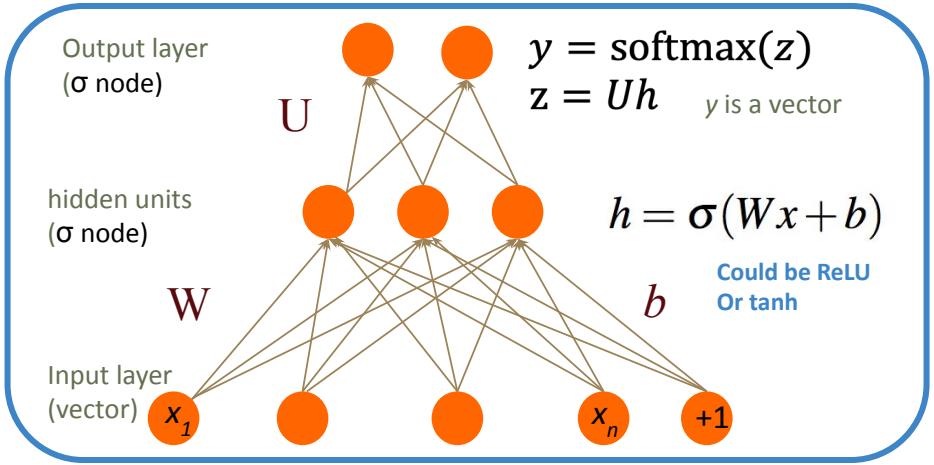
All pairs of attention scores!

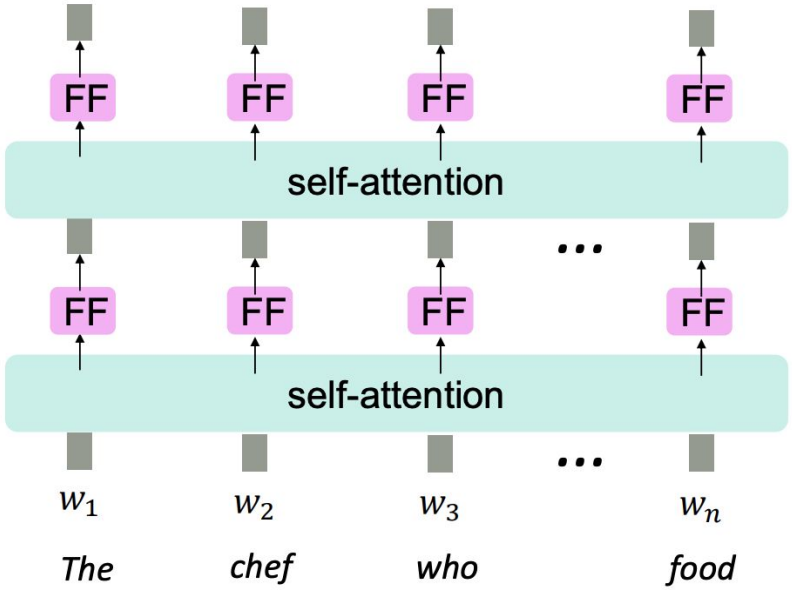
Next, softmax, and compute the weighted average with the values (with another matrix multiplication)



Adding Nonlinearities in Self-Attention

- **Problem:** there are no element-wise nonlinearities in self-attention; stacking more self-attention layers just re-averages value vectors
- **Solution:** add a **feed-forward network** to post-process each output vector.

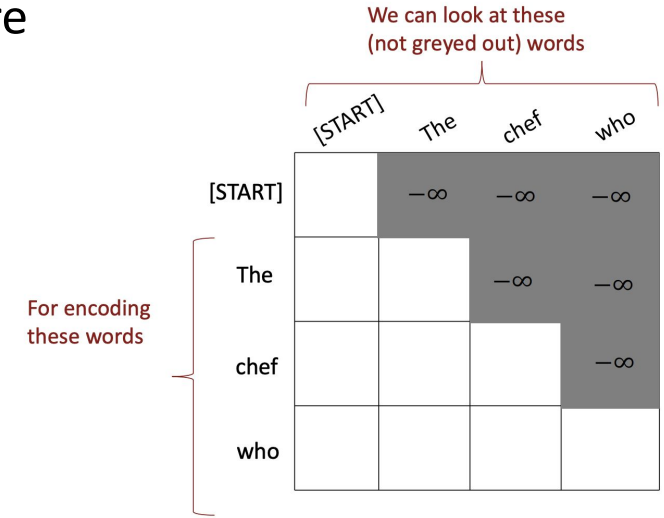




Intuition: the FF network processes the result of attention

Masking the Future in Self-Attention

- **Problem:** To use self-attention in **decoders**, we need to ensure we can't peek at the future when predicting a sequence during training
  - E.g. target sentence in translation
- **Possible solution?** At every timestep, we could change the set of **keys and queries** to include only past words. (Inefficient!)
- **Solution:** To enable parallelization, we **mask out attention** to future words by setting attention scores to $-\infty$.

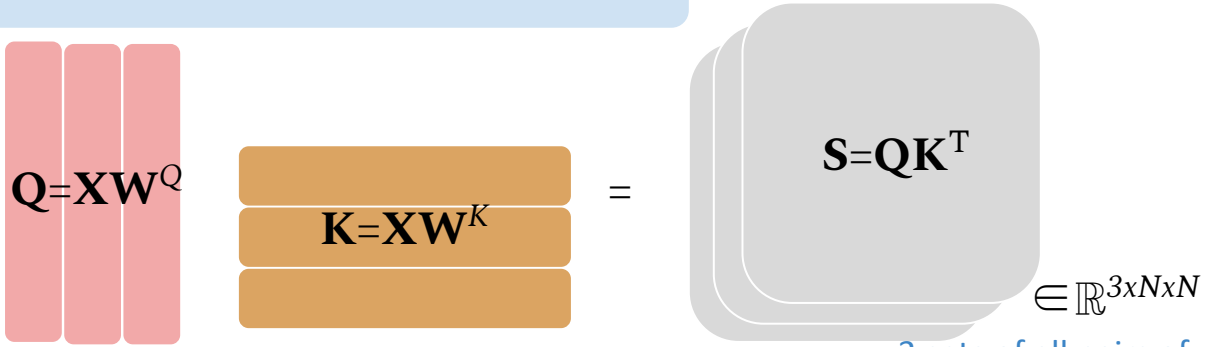


$$e_{ij}^{\text{mask}} = \begin{cases} \mathbf{q}_i \cdot \mathbf{k}_j, & j \leq i \\ -\infty, & j > i \end{cases}$$

Multihead Self-Attention: Visualization

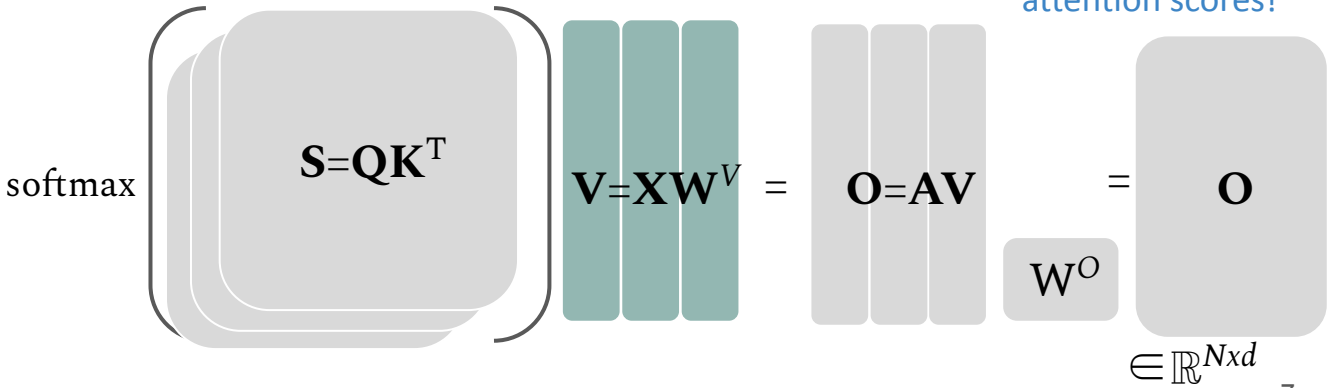
Still efficient: parallelizable

First, take the query-key dot products in one matrix multiplication



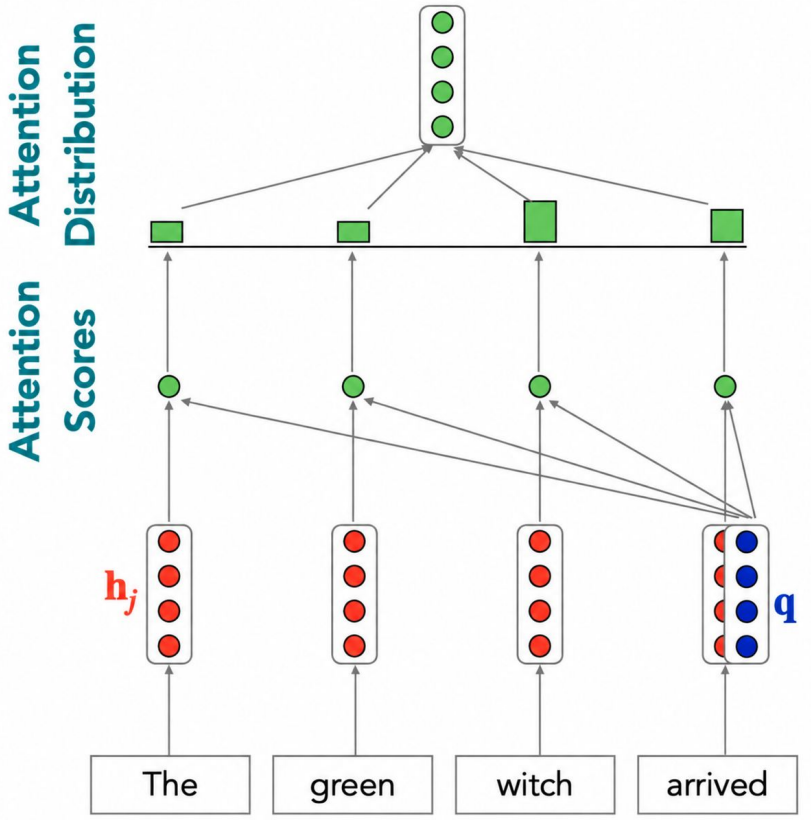
3 sets of all pairs of attention scores!

Next, softmax, and compute the weighted average with the values (with another matrix multiplication)



Self-Attention: Order Information?

- Self-attention networks are not typically based on Recurrent Neural Nets
  - No more order information!
- Since self-attention doesn't build in order information, we need to encode the order of the sentence in our keys, queries, and values.



Missing: Order Information

- Consider representing each sequence index as a vector
  - $\mathbf{p}_i \in \mathbb{R}^d$, for $i \in \{1, 2, \ldots, n\}$ are position vectors

- Easy to incorporate this info: just add the $\mathbf{p}_i$ to our inputs!
- Recall that $\mathbf{x}_i$ is the embedding of the word at index i. The positioned embedding is:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i + \mathbf{p}_i$$

- In deep self-attention networks, typical to do this at the first layer! You could concatenate them as well, but people mostly just add them.

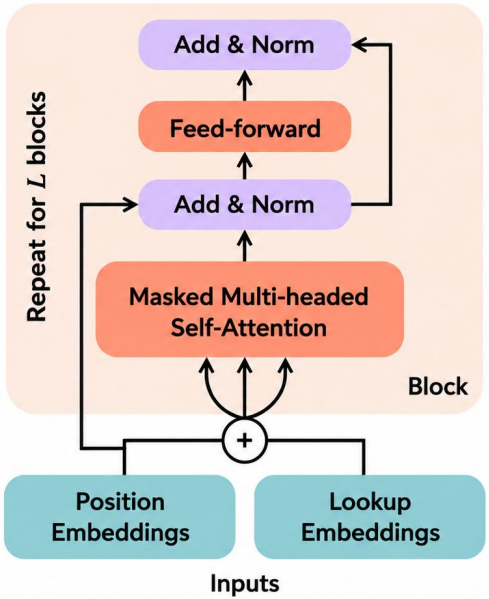
Positional Embeddings

- Maps integer inputs (for positions) to real-valued vectors
  - one per position in the entire context
- Can be randomly initialized and can let all $\mathbf{p}_i$ be learnable parameters (most common)
- Pros:
  - Flexibility: each position gets to be learned to fit the data
- Cons:
  - Definitely can't extrapolate to indices outside $1, \ldots, n$, where n is the maximum length of the sequence allowed under the architecture
  - There will be plenty of training examples for the initial positions in our inputs and correspondingly fewer at the outer length limits

Putting It All Together: Transformer Blocks

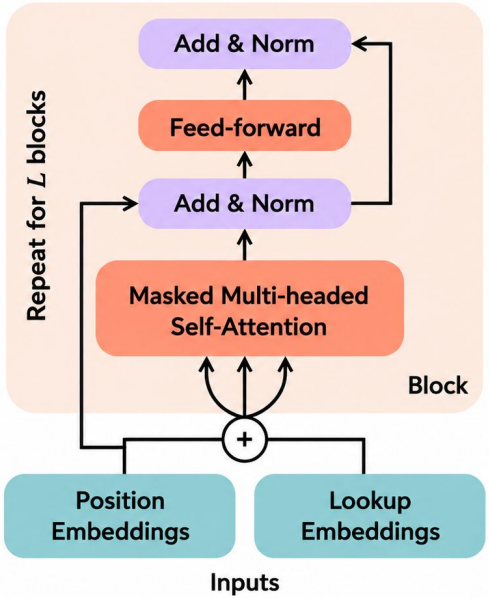
Self-Attention Transformer Building Blocks

- Self-attention:
  - the basis of the method; with multiple heads
- Position representations:
  - Specify the sequence order, since self-attention is an unordered function of its inputs.
- Nonlinearities:
  - At the output of the self-attention block
  - Frequently implemented as a simple feedforward network.
- Masking:
  - In order to parallelize operations while not looking at the future.
  - Keeps information about the future from "leaking" to the past.



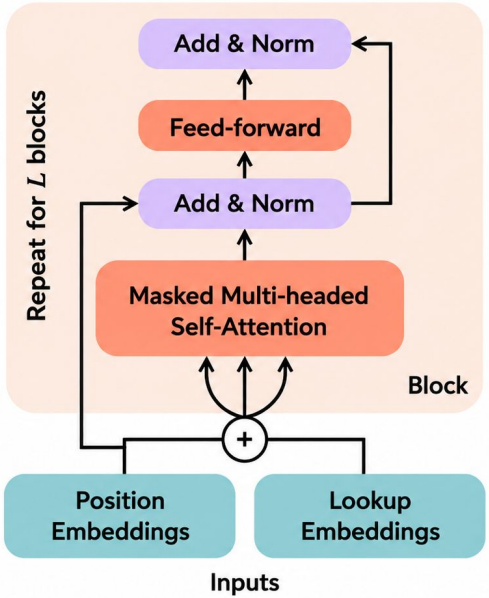
The Transformer Model

- Transformers are made up of stacks of transformer blocks, each of which is a multilayer network made by combining feedforward networks and **self-attention layers**, the key innovation of self-attention transformers
- The Transformer Decoder-only model corresponds to
  - a Transformer language model
- Lookup embeddings for tokens are usually randomly initialized



The Transformer Model

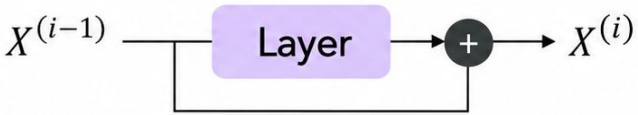
- Now that we've replaced self-attention with multi-head self-attention, we'll go through two **optimization tricks** that end up being :
  - **Residual Connections**
  - **Layer Normalization**
- In most Transformer diagrams, these are often written together as "Add & Norm"



Residual Connections



- Original Connections: $X^{(i)} = \text{Layer}(X^{(i-1)})$ where i represents the layer
- **Residual Connections**: trick to help models train better.
  - We let $X^{(i)} = X^{(i-1)} + \text{Layer}(X^{(i-1)})$
    - so we only have to learn "the residual" from the previous layer



- Allowing information to skip a layer improves learning and gives higher level layers **direct access to information** from lower layers (He et al., 2016).

Layer Normalization

- Layer normalization is another trick to help models train faster
- Idea: cut down on uninformative variation in hidden vector values by normalizing to zero mean and unit standard deviation **within each layer**
- Let $x \in \mathbb{R}^d$ be an individual (word) vector in the model.

Mean

$$\mu = \frac{1}{d}\sum_{j=1}^{d} x_j; \quad \mu \in \mathbb{R}$$

Standard deviation

$$\sigma = \sqrt{\frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2}; \quad \sigma \in \mathbb{R}$$

Result: New vector with zero mean and a standard deviation of one.

$$\hat{x} = \frac{x - \mu}{\sigma}$$

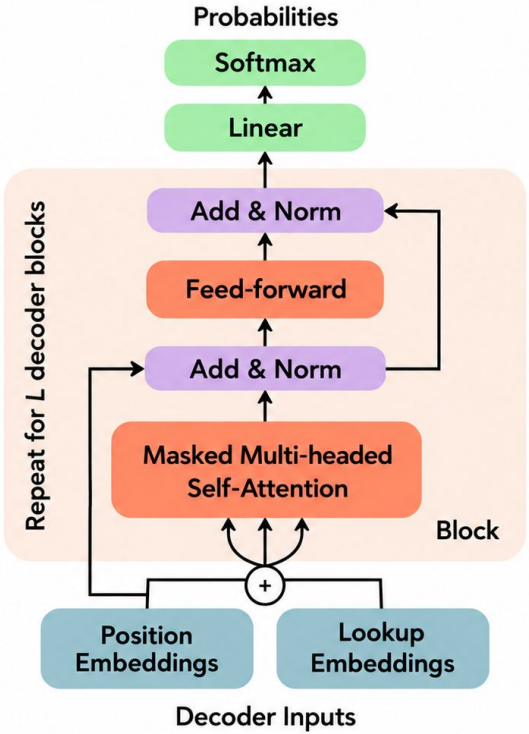
Component-wise subtraction

- Let $\gamma \in \mathbb{R}$ and $\beta \in \mathbb{R}^d$ be learned scale and shift parameters. (Can omit!)

$$\textbf{LayerNorm} = \gamma\hat{x} + \beta$$

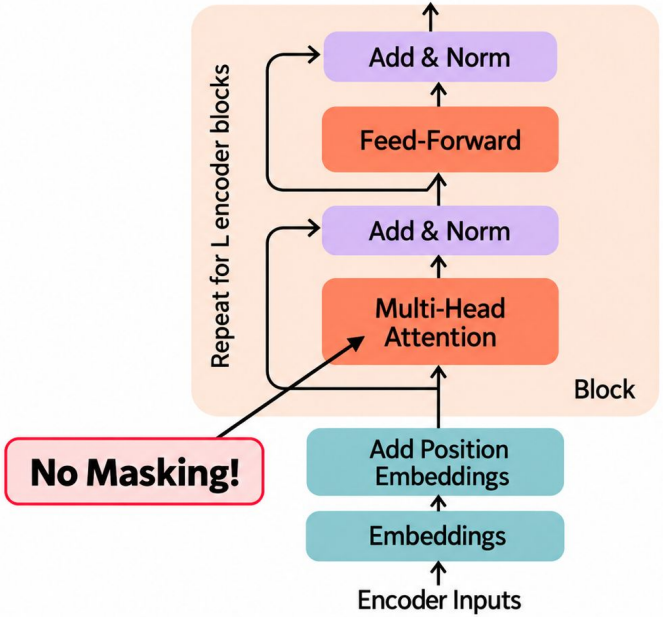
The Transformer Decoder

- The Transformer Decoder is a stack of Transformer Decoder **Blocks**.
- Each Block consists of:
  - Self-attention
  - Add & Norm
  - Feed-Forward
  - Add & Norm
- Output layer is as always a softmax layer
  - Sometimes called an **unembedding** layer



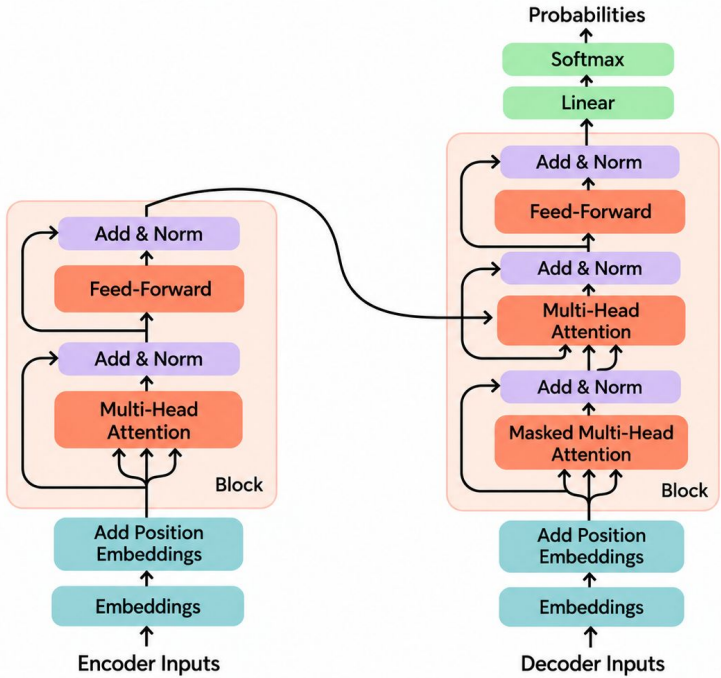
The Transformer Encoder

- The Transformer Decoder constrains to **unidirectional context**, as for **language models**.
- What if we want **bidirectional context**, like in a bidirectional RNN?
- This is the Transformer Encoder. The only difference is that we **remove the masking** in the self-attention.



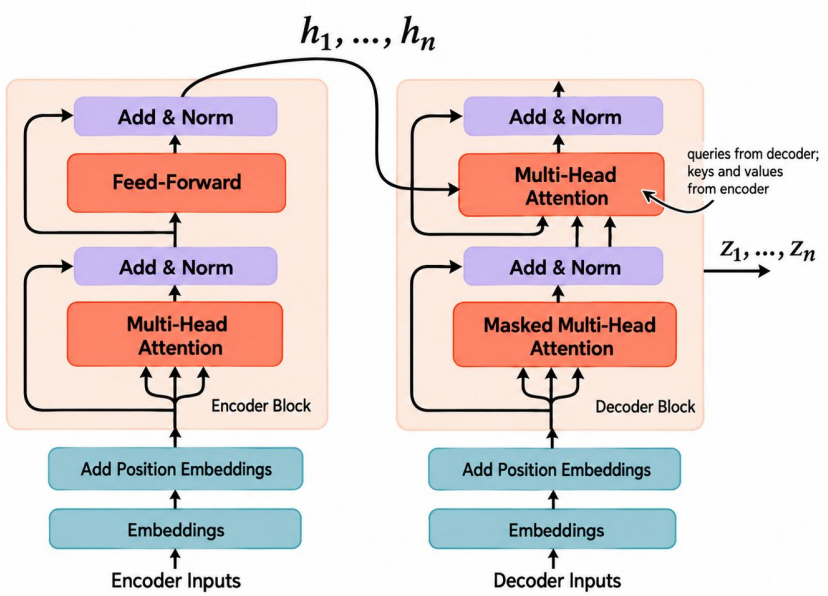
The Transformer Encoder-Decoder

- In seq2seq, we can process the source sentence with a **bidirectional** model and generate the target with a **unidirectional model**.
- For this kind of seq2seq format, we often use a Transformer Encoder-Decoder.
- We use a normal Transformer Encoder.
- Our Transformer Decoder is modified to perform **cross- attention** to the output of the Encoder.



Cross-Attention (details)

- We saw that self-attention is when keys, queries, and values come from the same source
- Let $\mathbf{h}_{1:M}$ be **output** vectors **from** the Transformer **encoder** $\mathbf{h}_j \in \mathbb{R}^{1\times d}$
- Let $z_{1:N}$ be input vectors from the Transformer **decoder** $\mathbf{z}_i \in \mathbb{R}^{1\times d}$
- Then keys and values are drawn from the encoder (like a memory)
  - $\mathbf{k}_j = \mathbf{h}_j \mathbf{W}^K$, $\mathbf{v}_j = \mathbf{h}_j \mathbf{W}^V$
- And the queries are drawn from the decoder, $\mathbf{q}_i = \mathbf{z}_i \mathbf{W}^Q$



Great Results with Transformers

First, Machine Translation from the original Transformers paper!

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

[Test sets: WMT 2014 English-German and English-French] [Vaswani et al., 2017]

Great Results with Transformers

Document generation!

Model	Test perplexity	ROUGE-L
seq2seq-attention, $L = 500$	5.04952	12.7
Transformer-ED, $L = 500$	2.46645	34.2
Transformer-D, $L = 4000$	2.22216	33.6
Transformer-DMCA, no MoE-layer, $L = 11000$	2.05159	36.2
Transformer-DMCA, MoE-128, $L = 11000$	1.92871	37.9
Transformer-DMCA, MoE-256, $L = 7500$	1.90325	38.8

[Liu et al., 2018]; WikiSum dataset

Great Results with Transformers

Document generation!

Model	Test perplexity	ROUGE-L
seq2seq-attention, $L = 500$	5.04952	12.7
Transformer-ED, $L = 500$	2.46645	34.2
Transformer-D, $L = 4000$	2.22216	33.6
Transformer-DMCA, no MoE-layer, $L = 11000$	2.05159	36.2
Transformer-DMCA, MoE-128, $L = 11000$	1.92871	37.9
Transformer-DMCA, MoE-256, $L = 7500$	1.90325	38.8

[Liu et al., 2018]; WikiSum dataset

- Before long, most Transformers results also included **pretraining**, a method we'll go over next!
- Transformers' parallelizability allows for efficient pretraining, and have made them the de-facto standard.

Motivating Model Pretraining from Word Embeddings

Motivating word meaning and context

Recall the quote we mentioned at the beginning of the course:

"You shall know a word by the company it keeps" (J. R. Firth 1957: 11)

This quote is a summary of distributional semantics, and motivated word2vec. But:

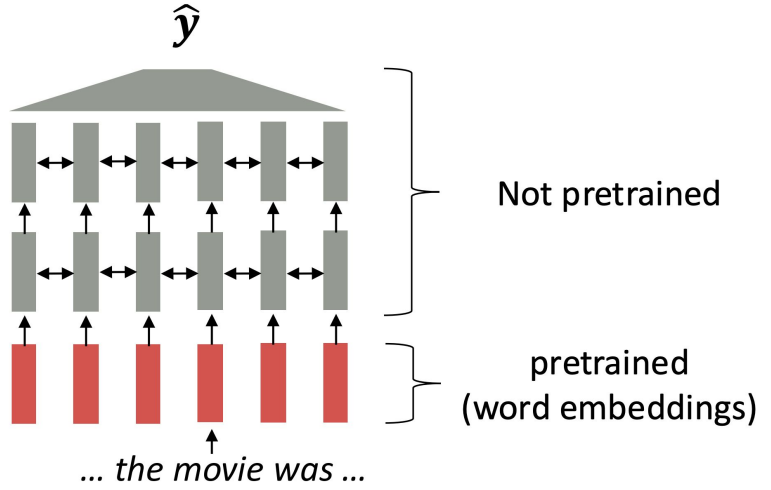
"… the complete meaning of a word is always contextual, and no study of meaning apart from a complete context can be taken seriously." (J. R. Firth 1935)

Consider *I* ***record*** *the* ***record***: the two instances of ***record*** mean different things

Pretrained Word Embeddings

Previously:

- Start with pretrained word embeddings
  - Word2vec
  - GloVe
  - Trained with limited context (windows)
- Learn how to incorporate context in an LSTM or Transformer while training on the task (e.g. sentiment classification)
- Paradigm till 2017

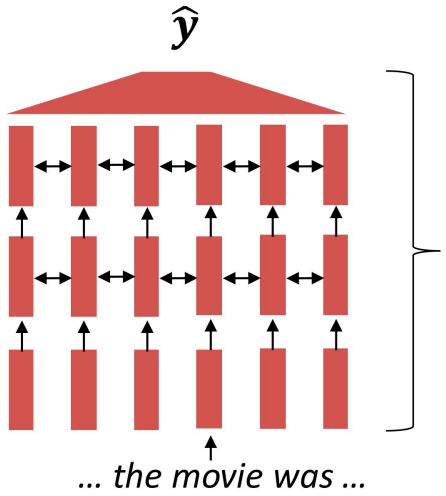


Recall, movie gets the same word embedding, no matter what sentence it shows up in

Pretrained Whole Models

In modern NLP:

- All (or almost all) parameters in NLP networks are initialized via pretraining.
- This has been exceptionally effective at building strong:
  - **representations of language**
  - **parameter initializations** for strong NLP models.
  - **probability distributions** over language that we can sample from



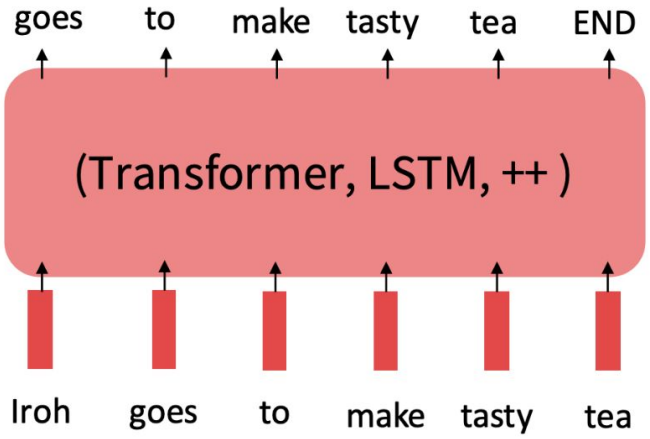
This model has learned how to represent entire sentences through pretraining.

Pretraining / Finetuning Paradigm

Pretraining can improve NLP applications by serving as parameter initialization.

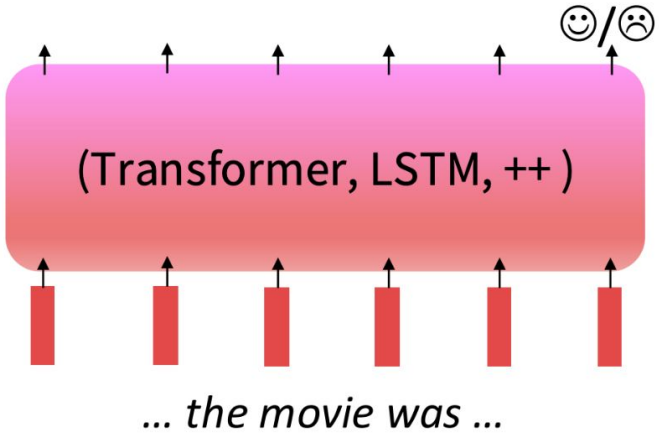
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on your task)

Not many labels; adapt to the task!



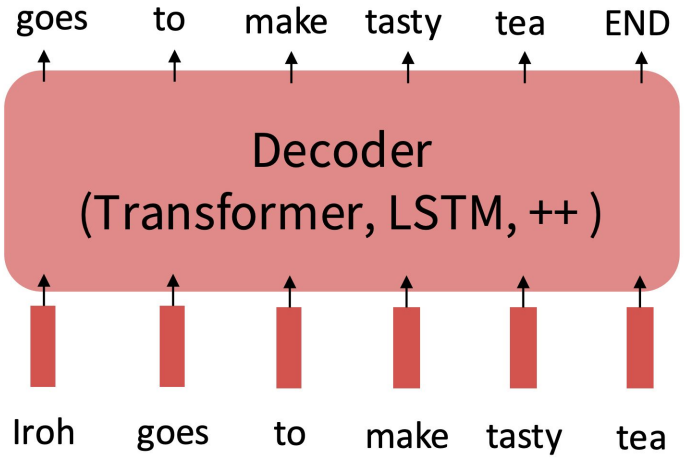
Pretraining: Intuition from SGD

Why should pretraining and finetuning help, from a "training neural nets" perspective?

- Consider, provides parameters $\hat{\theta}$ by approximating $\min_{\theta} \mathcal{L}_{\text{pretrain}}(\theta)$
  - (The pretraining loss.)
- Then, finetuning approximates $\min_{\theta} \mathcal{L}_{\text{finetune}}(\theta)$, starting at $\hat{\theta}$.
  - (The finetuning loss.)
- The pretraining may matter because stochastic gradient descent sticks (relatively) close to $\hat{\theta}$ during finetuning.
  - So, maybe the finetuning local minima near $\hat{\theta}$ tend to generalize well!
  - And/or, maybe the gradients of finetuning loss near $\hat{\theta}$ propagate nicely!

Pretraining Through Language Modeling

- Recall the language modeling task:
  - Model $p_\theta(w_t \mid w_{1:t-1})$, the probability distribution over words given their past contexts.
  - There's lots of data for this! (In English.)
- Pretraining through language modeling:
  - Train a neural network to perform language modeling on a large amount of text.
  - Save the network parameters.



Semi-supervised Sequence Learning

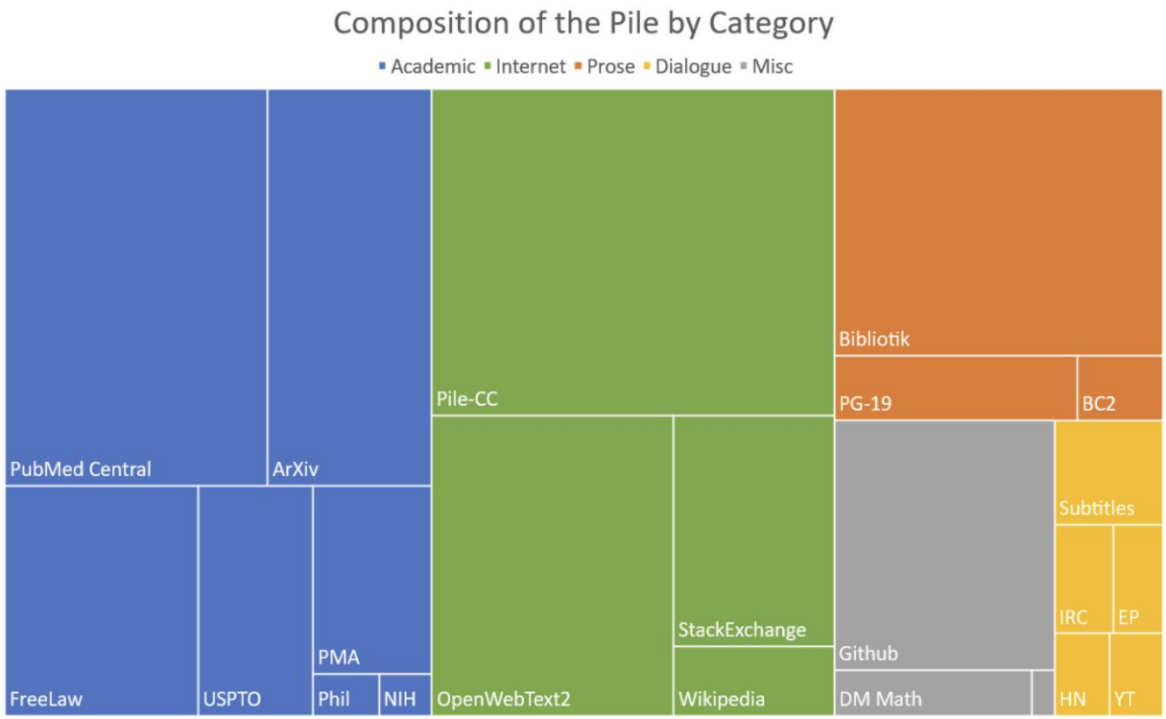
Andrew M. Dai
Google Inc.
adai@google.com

Quoc V. Le
Google Inc.
qvl@google.com

Dai and Le, 2015

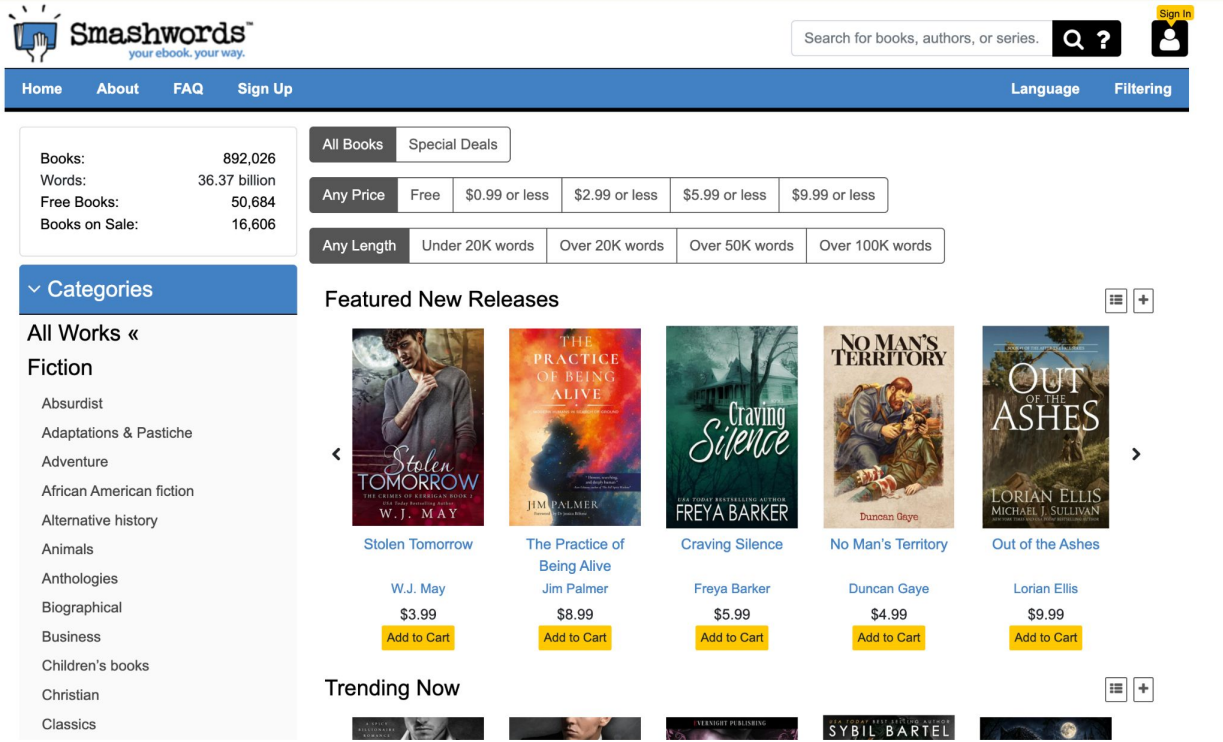
Pretraining Data

Where does the data come from?



Model	Training Data
BERT	BookCorpus, English Wikipedia
GPT-1	BookCorpus
GPT-3	CommonCrawl, WebText, English Wikipedia, and 2 book databases (“Books 1” and “Books 2”)
GPT-3.5+	Undisclosed

BookCorpus



- Contains around 7,000 self-published, free ebooks
- Controversial

Remember This From The First Class?

The New York Times Amends Lawsuit Against OpenAI and Microsoft

In a new court filing, The Times accused Microsoft of encouraging OpenAI to train its A.I. systems using copyrighted articles.

Listen · 4:04 min

Share full article



Privacy, Copyright Issues

Fair Use & Generative AI

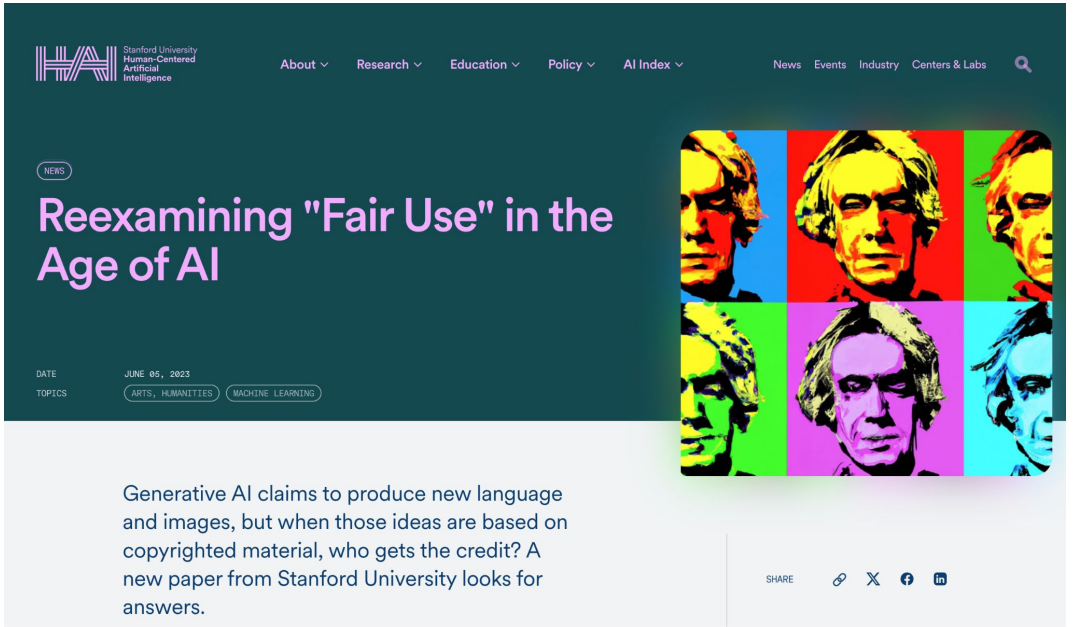
Google swallows 11,000 novels to improve AI's conversation

As writers learn that tech giant has processed their work without permission, the Authors Guild condemns 'blatantly commercial use of expressive authorship'



'It doesn't harm the authors' ... Google's headquarters in Mountain View, California. Photograph: Marcio Jose Sanchez/AP

https://www.theguardian.com/books/2016/sep/28/google-swallows-11000-novels-to-improve-ais-conversation



https://hai.stanford.edu/news/reexamining-fair-use-age-ai

Data Sources & Copyrighted Material


huggingface.co/datasets/monology/pile-uncopyrighted
0 6.48M
It is done, and submitted. You can play “Survival of the Tastiest” on Android, and on the web. Playing on the web…
{ "pile_set_name": "Pile-CC" }
<?xml version="1.0" encoding="UTF-8"?> <name>PD1</name> <description>Patient Additional…
{ "pile_set_name": "Github" }
Topic: reinvent midnight madness Amazon announced a new service at the AWS re:Invent Midnight Madness event. Amazon…
{ "pile_set_name": "Pile-CC" }
About Grand Slam Fishing Charters As a family owned business we know how important it is that your trip becomes the best…
{ "pile_set_name": "Pile-CC" }
Q: Why was Mundungus banned from the Hog's Head? In Order of the Phoenix while the trio were in the Hogs Head for the…
{ "pile_set_name": "StackExchange" }
Working Women, Special Provision and the Debate on Equality There has been considerable coverage in the media recently…
{ "pile_set_name": "Pile-CC" }
Previous 1 2 3 ... 8,918 Next

Pile Uncopyrighted

In response to authors demanding that LLMs stop using their works, here's a copy of The Pile with all copyrighted content removed.

Please consider using this dataset to train your future LLMs, to respect authors and abide by copyright law.

Creating an uncopyrighted version of a larger dataset (ie RedPajama) is planned, with no ETA.

Methodology

Cleaning was performed by removing everything from the Books3, BookCorpus2, OpenSubtitles, YTSubtitles, and OWT2 subsets.

Based on section 7.1 of the original paper, these datasets are the only ones which are not explicitly allowed to be used in AI training.

Data Sources & Copyrighted Material

dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research

Source	Doc Type	UTF-8 bytes *(GB)*	Documents *(millions)*	Unicode words *(billions)*	Llama tokens *(billions)*
Common Crawl	web pages	9,812	3,734	1,928	2,479
GitHub	code	1,043	210	260	411
Reddit	social media	339	377	72	89
Semantic Scholar	papers	268	38.8	50	70
Project Gutenberg	books	20.4	0.056	4.0	6.0
Wikipedia, Wikibooks	encyclopedic	16.2	6.2	3.7	4.3
Total		**11,519**	**4,367**	**2,318**	**3,059**

Table 1: The Dolma corpus at-a-glance. It consists of three trillion tokens sampled from a diverse set of domains; sourced from approximately 200 TB of raw text before curation down to an 11 TB dataset. It has been extensively cleaned for language model pretraining use. Tokens calculated using the LLaMA tokenizer.

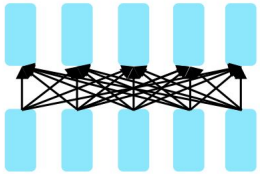
Copyright and fair use considerations.:

- "At the time of writing, the landscape governing applicability of copyright law and fair use doctrine (also known as "fair dealing") and language models is largely undetermined (Cooper et al., 2023; Lee et al., 2024)."
- "While most datasets we used were curated with copyright and licensing in mind (e.g., open access papers in peS2o (Soldaini and Lo, 2023), open source repositories in the Stack (Kocetkov et al., 2022)) or were already permissively licensed (e.g., Wikipedia is released under a Creative Commons license), we recognize that large web crawls may also contain copyrighted material. Yet, given current tools, it's not possible to reliably or scalably detect copyrighted materials in a corpus of this size."

Architectures (and Objectives) of Pretrained Transformer Models

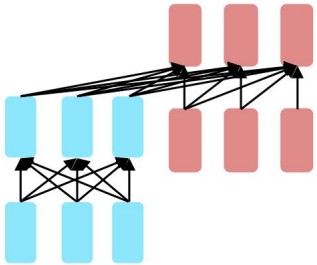
Pretraining for three types of architectures

The neural architecture influences the type of pretraining, and natural use cases.



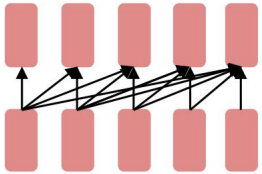
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



Encoder-Decoders

- Combines the encoder's bidirectional understanding of the input with the decoder's ability to generate output autoregressively.
- What's the best way to pretrain them?



Decoders

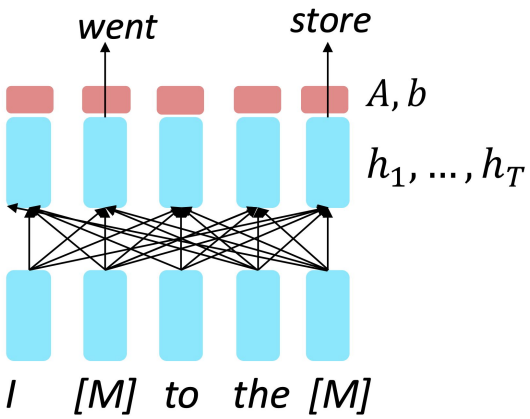
- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

Pretraining encoders: what pretraining objective to use?

So far, we've looked at language model pretraining. But **encoders get bidirectional context**, so we can't do language modeling!

Idea: replace some fraction of words in the input with a special [MASK] token; predict these words.

$$h_1, \dots, h_T = \text{Encoder}(w_1, \dots, w_T)$$
$$y_i \sim Aw_i + b$$



Only add loss terms from words that are "masked out." If $\tilde{x}$ is the masked version of x, we're learning $p_\theta(x|\tilde{x})$. Called **Masked LM**.

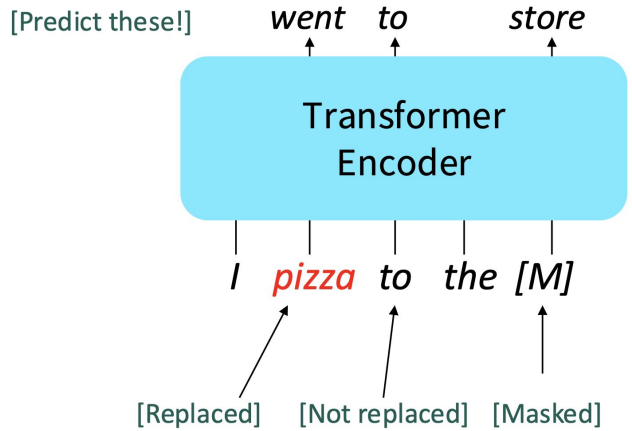
[Devlin et al., 2018]

BERT: Bidirectional Encoder Representations from Transformers

Devlin et al., 2018 proposed the "Masked LM" objective and **released the weights of a pretrained Transformer**, a model they labeled BERT.

Some more details about Masked LM for BERT:

- Predict a random 15% of (sub)word tokens.
  - Replace input word with [MASK] 80% of the time
  - Replace input word with a random token 10% of the time
  - Leave input word unchanged 10% of the time (but still predict it!)

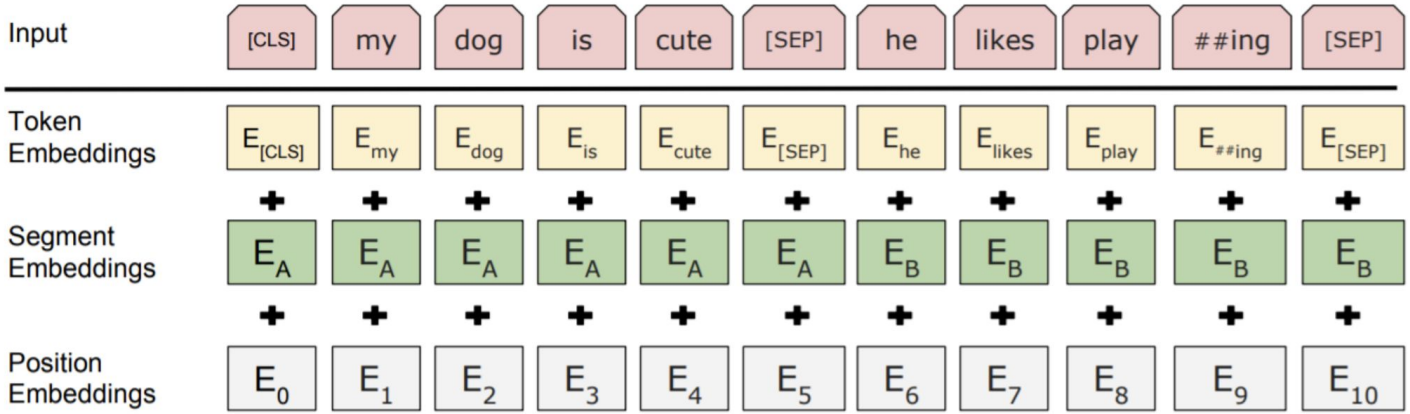


[Devlin et al., 2018]

Why? Doesn't let the model get complacent and not build strong representations of non-masked words. (No masks are seen at fine-tuning time!)

BERT: Bidirectional Encoder Representations from Transformers

- The pretraining input to BERT was two separate contiguous chunks of text:



- BERT was trained to predict whether one chunk follows the other or is randomly sampled.
  - • [CLS] and [SEP] tokens
  - • [SEP] is used for next sentence prediction - do these sentences follow each other?
  - • [CLS] for text classification

BERT: Bidirectional Encoder Representations from Transformers

Details about BERT

- Two models were released:
  - BERT-base: 12 layers, 768-dim hidden states, 12 attention heads, 110 million params.
  - BERT-large: 24 layers, 1024-dim hidden states, 16 attention heads, 340 million params.
- Trained on:
  - BooksCorpus (800 million words)
  - English Wikipedia (2,500 million words)
- Pretraining is expensive and impractical on a single GPU.
  - BERT was pretrained with 64 TPU chips for a total of 4 days.
  - (TPUs are special tensor operation acceleration hardware)
- Finetuning is practical and common on a single GPU
  - “Pretrain once, finetune many times.”



BERT: Bidirectional Encoder Representations from Transformers

BERT was massively popular and hugely versatile; finetuning BERT led to new state-of the-art results on a broad range of tasks.

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

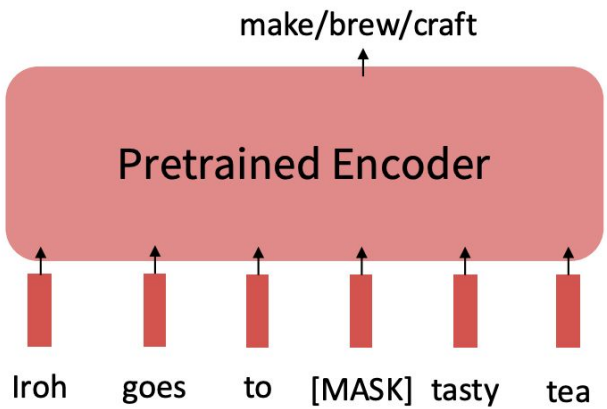
- **QQP:** Quora Question Pairs (detect paraphrase questions)
- **QNLI:** natural language inference over question answering data
- **SST-2:** sentiment analysis

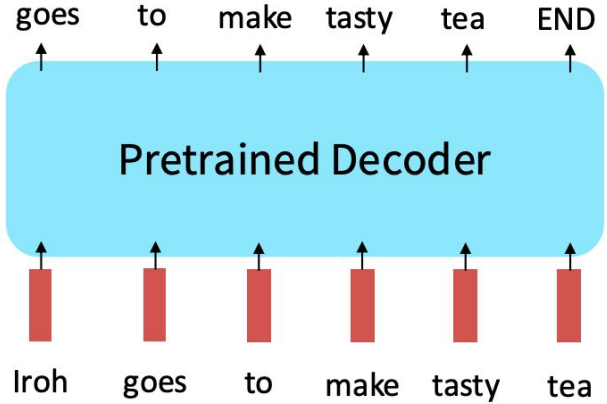
- **CoLA:** corpus of linguistic acceptability (detect whether sentences are grammatical.)
- **STS-B:** semantic textual similarity
- **MRPC:** microsoft paraphrase corpus
- **RTE:** a small natural language inference corpus

Limitations of Pretrained Encoders

Those results looked great! Why not used pretrained encoders for everything?

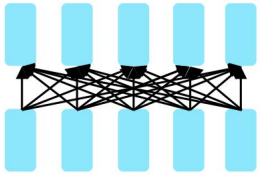
If your task involves generating sequences, consider using a pretrained decoder; BERT and other pretrained encoders don't naturally lead to nice autoregressive (1-word-at-a-time) generation methods.





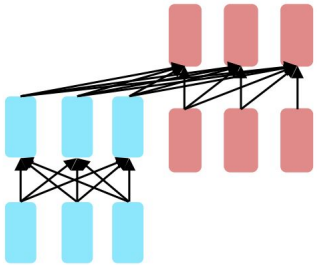
Pretraining for three types of architectures

The neural architecture influences the type of pretraining, and natural use cases.



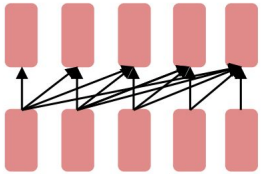
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



Encoder-Decoders

- Combines the encoder's bidirectional understanding of the input with the decoder's ability to generate output autoregressively.
- What's the best way to pretrain them?



Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

Pretraining decoders: what pretraining objective to use?

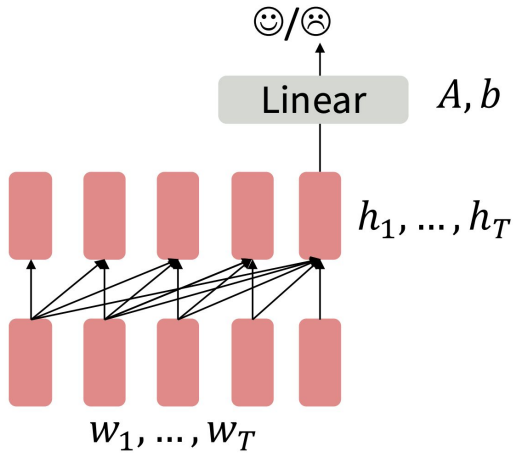
When using language model pretrained decoders, we can ignore that they were trained to model $p(w_t|w_{1:t-1})$.

We can finetune them by training a classifier on the last word's hidden state.

$$h_1, \dots, h_T = \text{Decoder}(w_1, \dots, w_T)$$
$$y \sim Ah_T + b$$

Where A and b are randomly initialized and specified by the downstream task.

Gradients backpropagate through the whole network.



[Note how the linear layer hasn't been pretrained and must be learned from scratch.]

Pretraining decoders: what pretraining objective to use?

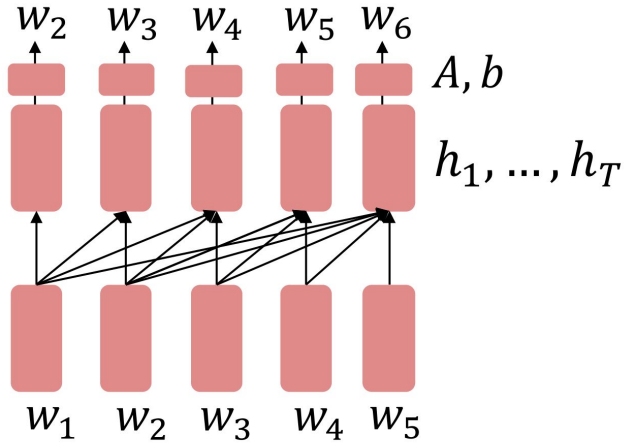
It's natural to pretrain decoders as language models and then use them as generators, finetuning their $p_\theta(w_t|w_{1:t-1})$.

This is helpful in tasks where the output is a sequence with a vocabulary like that at pretraining time!

- Dialogue (context=dialogue history)
- Summarization (context=document)

$$h_1, \ldots, h_T = \text{Decoder}(w_1, \ldots, w_T)$$
$$w_t \sim Ah_{t-1} + b$$

Where A and b are randomly initialized and specified by the downstream task.



[Note how the linear layer has been pretrained.]

Generative Pretrained Transformer (GPT)

2018's GPT was a big success in pretraining a decoder!

- Transformer decoder with 12 layers, 117M parameters.
- 768-dimensional hidden states, 3072-dimensional feed-forward hidden layers.
- Byte-pair encoding with 40,000 merges
- Trained on BooksCorpus: over 7000 unique books.
- Contains long spans of contiguous text, for learning long-distance dependencies.

[Radford et al., 2018]

Generative Pretrained Transformer (GPT)

How do we format inputs to our decoder for finetuning tasks?

Natural Language Inference (NLI): Label pairs of sentences as *entailing/contradictory/neutral*

Premise: *The man is in the doorway*
Hypothesis: *The person is near the door*
} **entailment**

Radford et al., 2018 evaluate on natural language inference.
Here's roughly how the input was formatted, as a sequence of tokens for the decoder:

[START] *The man is in the doorway* [DELIM] *The person is near the door* [EXTRACT]

The linear classifier is applied to the representation of the [EXTRACT] token.

Generative Pretrained Transformer (GPT)

GPT results on various *natural language inference* datasets.

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	89.3	-	-	-
CAFE [58] (5x)	80.2	79.0	89.3	-	-	-
Stochastic Answer Network [35] (3x)	80.6	80.1	-	-	-	-
CAFE [58]	78.7	77.9	88.5	83.3		
GenSen [64]	71.4	71.3	-	-	82.3	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	**61.7**
Finetuned Transformer LM (ours)	**82.1**	**81.4**	**89.9**	**88.3**	**88.1**	56.0

GPT-2

- GPT-2, a larger version (1.5B) of GPT trained on more data, was shown to produce relatively convincing samples of natural language.
- Moved away from classification, only generation

Context (human-written): In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.
GPT-2: The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved. Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

TODOs:

- **Homework 2 Due: 10/23**

Next Class: Guest Lecture - Scaling & Optimization